

ECG Signal Pre-Processing

Analysis Report

Name Rith Rajak

Roll No. 24116085

April 19, 2026

1 Introduction

Raw ECG signals from clinical recordings are almost never clean. There are two major sources of degradation that make it difficult to extract useful features from the signal directly. The first is broadband noise from muscle activity, electrode contact issues and powerline interference. The second is baseline wander – a very slow low-frequency drift usually caused by respiration or body movement. On top of this, detecting features like R-peaks from a noisy baseline-shifted signal is unreliable. The pipeline in this notebook deals with all three problems in sequence: denoising, baseline removal, and R-peak detection. This report analyses each stage – what it does, why the choices were made, and what the tradeoffs are.

The dataset used is the Shaoxing and Ningbo First Hospital 12-lead ECG database sampled at 500 Hz, 10 seconds per record (5000 samples per lead, 12 leads per record).

2 Stage 1: Denoising with Wavelet Transform

2.1 Choice of Method

The approach used here is the Discrete Wavelet Transform (DWT) with the **bior4.4** (Biorthogonal 4.4) wavelet. The core idea is that DWT decomposes the signal into approximation and detail coefficients at multiple frequency scales. Noise in ECG signals tends to sit at higher frequency scales, while the clinically meaningful waveforms – P wave, QRS complex, T wave – are at lower-to-mid frequency scales. By zeroing out the coefficients outside a chosen band and reconstructing, the noise is removed.

The reason bior4.4 specifically is chosen over something like db4 or haar is its **sym-metric shape**. Biorthogonal wavelets have linear phase response, which means the wavelet does not shift the waveform in time during reconstruction. For ECG this matters a lot – the timing of the R-peak relative to the P and T waves is clinically

significant. A non-symmetric wavelet can introduce subtle shifts that distort these relationships.

```

1 def denoise_signal(X, dwt_transform, dlevels, cutoff_low,
2   cutoff_high):
3     coeffs = wavedec(X, dwt_transform, level=dlevels)
4     for ca in range(0, cutoff_low):
5         coeffs[ca] = np.multiply(coeffs[ca], [0.0])
6     for ca in range(cutoff_high, len(coeffs)):
7         coeffs[ca] = np.multiply(coeffs[ca], [0.0])
8     return pywt.waverec(coeffs, dwt_transform)
9
10 signal_den = denoise_signal(signal, 'bior4.4', dlevels=9,
    cutoff_low=1, cutoff_high=7)

```

2.2 Parameter Analysis

The decomposition is done at **9 levels** and only levels 1 to 7 (inclusive) are kept. At 500 Hz, each decomposition level roughly halves the frequency range. The approximate frequency bands at each level are:

Level	Approx. Frequency Band (Hz)	Kept?
1 (approx)	0 – 0.98	No (zeroed)
2	0.98 – 1.95	Yes
...	...	Yes
7	31.25 – 62.5	Yes
8	62.5 – 125	No (zeroed)
9	125 – 250	No (zeroed)

The QRS complex has energy up to roughly 40–50 Hz. By keeping up to level 7 (about 62 Hz), the R-peak morphology is preserved. Zeroing level 8 and 9 removes the high-frequency noise. Zeroing level 1 (the approximation) removes the DC component and very low frequency content – this also partially addresses baseline wander, though not completely.

2.3 The Cutoff Tradeoff

The notebook explicitly notes that lowering `cutoff_high` removes more noise but risks losing R-peak morphology. This is a genuine tradeoff. Setting `cutoff_high = 5` instead of 7 would cut off at around 15–31 Hz. Since QRS energy extends up to 40 Hz, this would smooth out the sharp peak, reducing amplitude and widening the QRS complex in the reconstructed signal. In a classification task this is a serious

problem because the R-peak amplitude and QRS width are both diagnostic features. So 7 is a reasonable upper cutoff.

3 Stage 2: Baseline Wander Removal

3.1 Method

Baseline wander is removed using cascaded median filters. Two filters of width 200ms and 600ms (100 and 300 samples at 500 Hz) are applied one after the other. The result is an estimate of the baseline, which is then subtracted from the signal.

```
1 def get_median_filter_width(sampling_rate, duration):
2     res = int(sampling_rate * duration)
3     res += ((res % 2) - 1) # width must be odd for medfilt
4     return res
5
6 def filter_signal(X):
7     X0 = X
8     for mi in range(len(mfa)): # mfa = [100, 300] samples
9         X0 = medfilt(X0, mfa[mi])
10    return np.subtract(X, X0)
```

3.2 Why Median Filtering Works Here

A median filter with a window larger than one heartbeat period will not track the QRS complex (because the QRS is a sharp spike – the median across a 600ms window largely ignores it). What it does track is the slow baseline drift. So when you subtract the median-filtered signal from the original, you are left with the cardiac activity without the drift. The 200ms filter handles shorter duration wander and the 600ms filter handles longer. Cascading them is more effective than a single wide filter.

A key point is that the filter width needs to be **longer than the P-R interval and T-wave duration** but not so long that it starts tracking real signal changes. At 600ms and 500 Hz (300 samples), this condition is satisfied for typical resting heart rates (60–100 bpm, cycle length 600–1000 ms).

3.3 Order of Operations: Denoise First or Baseline First?

The notebook explicitly tests both orderings:

```
1 s1 = denoise_signal(filter_signal(signal), 'bior4.4', 9, 1, 7) #
    baseline -> denoise
```

```

2 s2 = filter_signal(denoise_signal(signal, 'bior4.4', 9, 1, 7)) #
   denoise -> baseline

```

The result is that **denoise first gives a cleaner output**. The reason makes sense in hindsight: if baseline wander is removed first on a noisy signal, the median filter is working on a signal that has noise affecting each sample. The noise means the median values are slightly off, so the baseline estimate is noisier. When you then apply the wavelet denoiser, it has to deal with both the original high-frequency noise and the residuals left from an imperfect baseline subtraction. Denoising first gives the baseline filter a much cleaner input, so its estimate of the drift is more accurate.

4 Stage 3: R-Peak Detection

4.1 Algorithms

Six R-peak detectors from the `py-ecg-detectors` library were tested on the same preprocessed signal:

- `pan_tompkins_detector` – Bandpass filter + derivative + squaring + moving integration + adaptive threshold. Classic 1985 algorithm, still widely used as a baseline.
- `hamilton_detector` – Similar to Pan-Tompkins with some modifications to the thresholding logic.
- `christov_detector` – Uses adaptive threshold that adjusts based on recent peak amplitudes. More robust to amplitude variation.
- `engzee_detector` – Derivative-based with a different threshold strategy.
- `swt_detector` – Uses Stationary Wavelet Transform (SWT) instead of band-pass filtering to locate the QRS.
- `two_average_detector` – Uses two moving averages at different timescales.

```

1 for kd in detectors:
2     r_peaks = (np.array(
3         detectors[kd][0](np.hstack((signal_pad, signalflt)))
4     ) - signal_pad_samples)
5     detectors[kd][1] = r_peaks

```

A small zero-pad of 10 samples is prepended before detection. This is done because several detectors have a startup period – their internal buffers or adaptive thresholds need a few samples to initialise properly, and without the pad they can miss the very first peak.

4.2 Results and Analysis

The `christov_detector` was found to perform best for most records. This is consistent with its design: the adaptive thresholding means the detector adjusts to the actual amplitude of the peaks it is seeing, rather than using a fixed threshold. In a real-world dataset like Shaoxing-Ningbo, patient-to-patient amplitude variation is significant, and this makes fixed-threshold methods like Pan-Tompkins less reliable.

The performance also varies by lead. Lead II is typically preferred for R-peak detection because the QRS complex has the largest amplitude in this lead due to the axis of the lead aligning with the mean cardiac electrical axis. Leads like aVR or V1 can have biphasic or inverted QRS morphology, which confuses many detectors that assume a predominantly positive peak.

The SWT detector (`swt_detector`) is theoretically appealing because it uses the same wavelet framework as the denoising step, but in practice it did not consistently outperform Christov on this dataset. This could be because the SWT approach is more sensitive to the residual noise still present after preprocessing.

5 Analysis: Testing Unlabelled Beats Against Prior Models

The notebook notes that beats in this dataset are not individually segmented or labeled, and suggests testing with previous models and manually asserting results. This section analyses what that process would actually look like and what problems it would run into.

5.1 Beat Segmentation

Before any model can be applied to individual beats, the R-peak locations found by the Christov detector need to be used to segment the continuous signal into individual heartbeat windows. The standard approach is to take a fixed window around each detected R-peak – typically 200ms before and 400ms after (at 500Hz, that is 100 samples before and 200 samples after, giving a 300-sample beat window). This is called R-peak centred segmentation.

```
1 beats = []
2 for peak in data_peaks:
3     if peak - 100 >= 0 and peak + 200 <= len(data_signal):
4         beat = data_signal[peak - 100 : peak + 200]
5         beats.append(beat)
6 beats = np.array(beats) # shape: (num_beats, 300)
```

If the R-peaks are wrong (missed peaks or false detections from the Christov detector), the segmented beats will be misaligned. A misaligned beat fed into a classifier will almost certainly be misclassified, regardless of the model quality.

5.2 What the Model Would See

If a previously trained model is applied to these segments, there are a few things to expect. First, the model's training data matters: if it was trained on a different database (e.g. MIT-BIH Arrhythmia Database) with different preprocessing, the input distribution will be different from what the model expects. The Shaoxing-Ningbo dataset has 12 leads whereas MIT-BIH has 2. If the model was trained on lead II only, applying it to other leads will likely give degraded accuracy.

Second, the preprocessing pipeline here (bior4.4 DWT + median filter baseline removal) may differ from whatever preprocessing was used when the model was trained. If the training data was filtered differently, the beat morphology presented to the model will be subtly different, and classification confidence will drop even if the beats are technically the same arrhythmia type.

5.3 Manual Assertion

Manually asserting the results means comparing the model's predicted labels against the ground truth from the `.hea` file (`Dx` field). There are two issues with this. The `Dx` field gives a record-level diagnosis (e.g. "Atrial Fibrillation"), not a beat-level label. So you cannot directly verify whether each individual beat was correctly classified – only whether the model's outputs are consistent with the overall diagnosis.

If, say, the `Dx` field says "Normal Sinus Rhythm" and the model classifies most beats as normal but flags 2–3 beats as ectopic, that could be a model error or it could be genuine ectopic beats within an otherwise normal record. You cannot tell without an annotated beat-level label, which this dataset does not provide at that granularity.

5.4 Expected Outcome

If the Christov detector is accurate and the beat segmentation is clean, a reasonably trained classifier should be able to roughly agree with the record-level `Dx` in the majority of cases. For normal sinus rhythm records this is most reliable. For complex arrhythmias (e.g. Atrial Fibrillation, where there are no well-defined R-peaks and the rhythm is irregular), both the R-peak detector and any downstream beat classifier will struggle. AF in particular breaks the assumption of periodic, morpho-

logically consistent beats that most detectors and classifiers are built around.

In summary, manual assertion against the Dx labels would give a rough sanity check but not a rigorous evaluation. A proper evaluation would need beat-level annotations or at least the full confusion matrix at the record level across all 12 leads, which is more work than the notebook currently sets up for.

6 Overall Assessment

The pipeline is a reasonable baseline for ECG preprocessing. The wavelet denoising with bior4.4 is a well-established approach and the parameter choices (9 levels, cut-offs 1–7) are sensible for 500 Hz data. The cascaded median filter baseline removal is straightforward and effective. Doing denoising before baseline removal is the correct order and the notebook experimentally confirms this rather than just assuming it.

The main gap is that detection accuracy is only assessed visually on a few records. For a proper evaluation you would want to run this pipeline on a larger set of records, quantify missed peaks and false detections, and compare detectors more systematically. The Christov detector's advantage over Pan-Tompkins and others might not hold uniformly across all lead types or arrhythmia classes.